

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 12/Unit 12:

Constant Correctness and Static Functions

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Introduction
2. What is constant correctness?
3. Some Cases
4. Static Functions
5. Summary



Constant Correctness

- In C++, constant correctness refers to using the `const` keyword to enforce that certain data cannot be modified after its definition.
- This helps avoid accidental changes to values that are meant to remain constant.
- Code is more predictable and potential bugs are reduced.
- Constant correctness is commonly applied to variables, pointers, and member functions.



Declaring const variables

- Declaring a variable with `const` prevents it from being modified after initialization.
- Example:

```
const int maxSize = 100;
```

```
// maxSize = 200;
```

```
// Error: maxSize is constant and cannot be changed
```

Declaring const pointers

- In C++, pointers can be made constant in two ways:
- **Pointer to a constant:** You cannot modify the value being pointed to.

```
const int* ptr = &maxSize;
```

```
// ptr points to a constant
```

```
int* ptr = 10; // Error: value cannot be changed
```

- **Constant pointer:** The pointer cannot change to point to a different address

```
int value = 5;
```

```
int* const ptr = &value;
```

```
// ptr cannot point to another address
```

```
ptr = &anotherValue;
```

```
// Error: pointer cannot be changed
```



Combining the previous 2 cases

- Constant pointer to a constant:
- Neither the address nor the value being pointed to can be changed.

```
const int* const ptr = &maxSize;  
// ptr is a constant pointer to a constant int
```



Constant Member Functions

- Declaring a member function with `const` prevents it from modifying any member variables of the class.

```
class MyClass {  
    public:  
        int getValue() const { return value; }  
        // Constant member function  
  
        void setValue(int newValue) { value =  
newValue; }  
    private:  
        int value;  
};
```



Benefits of Constant Correctness

- Ensures **Integrity**: Guarantees that certain parts of your program state cannot be altered after being set.
- Enhances **Code Clarity**: Makes it clear to programmers which values are expected to remain constant.
- Enables **Optimization**: Allows compilers to perform optimizations since they know that certain values won't change.

Using constant correctness can improve both the reliability and efficiency of your C++ code.



What about `const` parameters?

- In C++, function parameters can be marked as `const` to enforce constant correctness.
- This indicates that the parameter will not be modified within the function body,
- It also makes the function safer and potentially more efficient.
- Constant parameters are especially useful for reference and pointer parameters, where the data being referred to might be changed unintentionally if not marked `const`.



const parameter with primitive types

- For basic data types like int or double, marking a parameter as const doesn't offer much benefit
- This is because these parameters are passed by value
- Pass by value means a copy is made.

```
void printValue(const int value) {  
    // value = 10;  
    // Error: cannot modify a constant parameter  
    std::cout << value << std::endl;  
}
```



const parameter with reference types

- For larger objects, passing by reference (with const) is more efficient than passing by value.
- This avoids copying the object, while const ensures the object won't be modified.

```
void display(const std::string& message) {  
    // message += " - additional text";  
    // Error: message is constant  
    std::cout << message << std::endl; }  

```

- Using const here saves the cost of copying a potentially large object while ensuring the function doesn't alter message.



Constant Pointer Parameters

- When using pointers, marking a parameter as `const` is valuable because it clarifies whether the function will modify the data being pointed to.
- **Pointer to Constant:** The data pointed to cannot be modified, but the pointer itself can be reassigned.

```
void processArray(const int* arr) {  
    // *arr = 10;  
    // Error: cannot modify the value pointed to  
    // arr = nullptr;  
    // OK: pointer itself can be changed  
}
```



Constant Pointer Parameters

- **Constant Pointer to Constant:** Neither the pointer nor the data it points to can be modified.

```
void processFixedArray(const int* const arr) {  
    // *arr = 10;  
    // Error: cannot modify the value pointed to  
    // arr = nullptr;  
    // Error: cannot modify the pointer itself  
}
```



Constant Parameters in Function Declarations

- In function declarations (prototypes), marking parameters as `const` helps clarify intentions for both the programmer and the compiler. For example:
- **`void compute(const int a, const std::vector<int>& data);`**

Some interesting cases

- Have a look at the following code:

```
void g1(std::string& s);
void f1(const std::string& s)
{
    g1(s);
    // Compile-time Error since s is const
    std::string localCopy = s; //creates a deep copy
    g1(localCopy);
    // Okay since localCopy is not const
}
```



Some interesting cases (Questions)

- What does "const X* p" mean?
- What's the difference between "const X* p", "X* const p" and "const X* const p"?
- What does "const X& x" mean?
- What do "X const& x" and "X const* p" mean?
- Does "X& const x" make any sense?

Answers at:

<https://isocpp.org/wiki/faq/const-correctness#overview-const>



Static Variables – Review

- Recall that we have studied static variables earlier in the course.
- A static variable (also called a class variable) maintains only one copy for the entire class.
- In other words, each object of a class does not have a separate copy of static variable.
- Instead all objects of the class share the same copy of the static variable.



Static Functions

- In C++, a static function is a function that belongs to a class but does not operate on a specific instance of that class.
- Static functions are declared with the `static` keyword and have unique properties and use cases.



Properties of a Static Function

- Static functions belong to the class itself, not to any particular object.
- They can be called directly on the class without creating an instance of the class.

```
class MathUtils {  
    public:  
        static int add(int a, int b) {  
            return a + b;  
        }  
};  
int result = MathUtils::add(5, 3);  
// No object needed to call add()
```



Access Restrictions

- Static functions can only access other static members (vars/functions) of the class.
- They do not have access to `this`, as they do not belong to any specific instance.

```
class Counter {  
    private:  
        static int count; // Static member variable  
    public:  
        static void increment() {  
            count++; // Can access static variable  
            // nonStaticVariable++;  
            // Error: cannot access non-static members  
        }  
};
```

```
int Counter::count = 0; // Initialize static member outside class
```



Cannot be Virtual

- Static functions cannot be virtual, meaning they do not support polymorphism or dynamic dispatch.
- This is because virtual functions work with instances of a class, while static functions are tied to the class itself.



Use Cases

- Static functions are often used for utility functions that don't need an object to work.
- Common examples include mathematical calculations, logging functions.

```
class Logger {  
public:  
    static void log(const std::string& message) {  
        std::cout << "Log: " << message << std::endl;  
    }  
};  
Logger::log("This is a message.");  
// Can call log without a Logger object
```



Static Functions

- Static functions are useful when a function's logic is specific to the class but doesn't need to interact with the instance's state.
- This can help encapsulate logic within the class itself, rather than defining the function globally.
- Static functions have a **lifetime** independent of any class instances.
- They exist as long as the program runs, and they don't require any instance initialization to be accessible.



Example: Factory design pattern

- A common use of static functions is in implementing the Factory Design pattern.
- Here, a static function creates instances of a class without requiring an instance of that class.

```
class Product {  
public:  
    static Product createDefault() {  
        // Create a default Product instance  
        return Product();  
    }  
};  
Product defaultProduct = Product::createDefault();  
// Create a Product instance using static factory method
```



Summary

- Const correctness is an important aspect of type correctness in C++.
- There are several variations of const correctness, each with its own intricacies.
- Static functions, just like static variables, are class functions.
- Static functions are not tied to specific objects. Rather they are linked with the class instances.